Final Phase

Phase 5

Decision Making Through Political Letters Using NER

Table of Contents

Introduction 3
Experimental Setup 3
Data Preprocessing 5
Model Architecture 6
Training Procedures 9
Evaluation Metrics 9
Transition to Implementation Phase 11
Computational Environment 12
Data Ingestion 12
Model Initialization 12
Training and Evaluation 15
Testing and Evaluation 16
Model Saving 17
Interface and Deployment 17
Results Analysis 20
Summary Table of Evaluation Metrics: 20
Conclusion 21
Future Work 21
References *(APA)* 22
Appendix 23

Introduction

This report presents the development and evaluation of a Named Entity Recognition (NER) system tailored for Arabic political texts using the AraBERT model. The core objective of the project is to automate the extraction of key entities such as persons, organizations, locations, and miscellaneous references within political discourse, thereby enhancing the efficiency and accuracy of decision-making processes in governmental and diplomatic domains.

The project follows a comprehensive approach that includes both practical implementation and forward-looking planning. It details each step of the system development—from data preprocessing and model training to performance evaluation and user interface integration. Additionally, it explores future directions for expanding the system's capabilities, including the application of deep learning to analyze the tone and intent of political messages and the use of natural language generation to produce appropriate automated responses.

Through this report, readers will gain insight into the technical foundations of the project, its real-world applications, and the strategic vision for its continued evolution in the field of political language processing.

Experimental Setup

In this project, we aim to improve decision-making through political speeches using Named Entity Recognition (NER).

- The Arabic dataset consists of political speeches, articles, and official correspondence collected from various sources such as Kaggle and GitHub. These texts are stored in CSV format and have been preprocessed to ensure the integrity, cleanliness, training readiness, and consistency of the data.

- For entity recognition, we use AraBERT, a pre-trained, transformer-based model specialized in Arabic texts. To customize the model to our specific domain, we fine-tune it on our selected dataset to extract key entities such as names, agreements, and organizations.

- We will build our code using Python on Google Colab, using the PyTorch framework, which provides flexibility for fine-tuning transformer models. Training and inference are performed on the CPU, as GPU limitations hinder visualization of the results.

- To evaluate the performance of our model, we use standard NER evaluation metrics, including:

Accuracy: Measures the proportion of correctly identified entities out of all identified entities.

Recall: Measures the number of entities successfully identified.

F1 score: The harmonic mean of precision and recall, providing a balanced assessment of model performance.

These metrics ensure an accurate assessment of the quality of entity extraction and allow us to analyze how well our improved AraBERT model adapts to political text data.

Candidate computing resources

In the future implementation phase, the model will be developed using both a local Lenovo i7 laptop with 16 GB RAM and Google Colab. Initial data preprocessing and testing will be conducted locally using Python 3.10 and essential libraries such as pandas, numpy, and transformers. For training and evaluation, Google Colab will be used with a CPU runtime (Intel Xeon, ~12.7 GB RAM) due to GPU limitations. The environment will include torch, arabert, seqeval, and other necessary tools to fine-tune AraBERT and evaluate NER performance efficiently.

A user scenario using our system is mentioned to illustrate its importance and workflow:

Scenario: A Foreign Ministry Employee Using Our System

Ahmed, an employee at the Ministry of Foreign Affairs, receives hundreds of diplomatic messages daily from embassies and consulates, making manual processing overwhelming.

- The AI system automatically analyzes messages, extracts key names, agreements, and main topics, then determines the required action, such as following up on an agreement or documenting a specific discussion.
- A new report is arriving about a diplomatic meeting. The system identifies relevant figures, countries, and agreements, then suggests appropriate action, like documenting the agreement or initiating a follow-up.
- The system presents a clear summary to help Ahmed take the next step without reading the full report.
- Outcome: Reduced manual effort, faster information processing, and improved documentation accuracy.

Data Preprocessing

In this project, the dataset undergoes a series of preprocessing steps to ensure it is suitable for training the AraBERT model for Named Entity Recognition (NER). The following steps were carried out:

Data Loading

The dataset was uploaded from hugging face: Upload dataset · iSemantics/conllpp-ner-ar at 48255f0, this dataset includes labeled tokens and entity tags that we can use for model training.

1. **Text cleaning**

   Special characters, punctuation marks, and unwanted symbols are removed from the text. As well as removing extra white spaces.

2. **Tokenizing**

The text is divided into smaller components known as tokens, which can be words or subword units. This process is especially crucial for Arabic due to the language's intricate morphology. To manage the complexities of Arabic text, the AraBERT tokenizer is employed, enabling the division of words into tokens that the model can effectively process.

Subword Tokenization: The tokenizer facilitates the division of words into subword segments, enabling the model to manage out-of-vocabulary terms by identifying shared subword elements. This approach is especially beneficial for languages such as Arabic, where words can exhibit numerous variations based on their root.

3. **Entity labeling**

Each token is labeled using the **BIO format** (Begin, Inside, Outside), indicating whether it is part of an entity or not:

- B-PER: Beginning of a person's name.
- I-PER: Inside a person's name.
- B-ORG: Beginning of an organization name.
- I-ORG: Inside an organization name.
- B-LOC: Beginning of a location name.
- I-LOC: Inside a location name.
- O: Outside of any entity.

4. **Train-Test split**

Training set: 10,250 examples

Validation set: 2,383 examples

2,572 examples :**Test set**

This split ensures that the model can be trained ,validated ,and tested on separate data to evaluate its performance accurately.

5. **Padding and Truncation**
   - **Padding** :Shorter sequences are padded to ensure uniform sequence lengths across the dataset.
   - **Truncation** :Longer sequences are truncated to a fixed maximum length , ensuring all inputs are compatible with the model.

6 .Attention Masking

Attention masks are designed to distinguish between actual tokens and padding tokens .The model utilizes these masks to concentrate solely on the relevant tokens while disregarding the padded areas during processing.

Model Architecture

For this task, we selected the AraBERT model due to its effectiveness in handling Arabic language tasks, particularly in Named Entity Recognition (NER) applications. AraBERT, which is a transformer-based model pre-trained on large Arabic corpora, leverages the power of deep learning to capture context-rich information, making it highly suitable for understanding the complex syntactic and semantic structures of the Arabic language. Its proven ability to provide powerful word embeddings has made it the go-to choice for a variety of Arabic language processing tasks.

Why AraBERT?

Arabic, as a morphologically rich and syntactically complex language, presents significant challenges for traditional NLP models. The use of standard models on Arabic text often leads to difficulties in understanding word meanings, sentence structures, and context. AraBERT was specifically fine-tuned to address these challenges and has shown state-of-the-art performance for Arabic NER tasks. The bidirectional attention mechanism, which is a core component of the transformer architecture, allows AraBERT to capture context at various levels—this is especially important in political NER tasks, where correctly identifying entities such as people, organizations, and locations in the context of political discourse is critical.

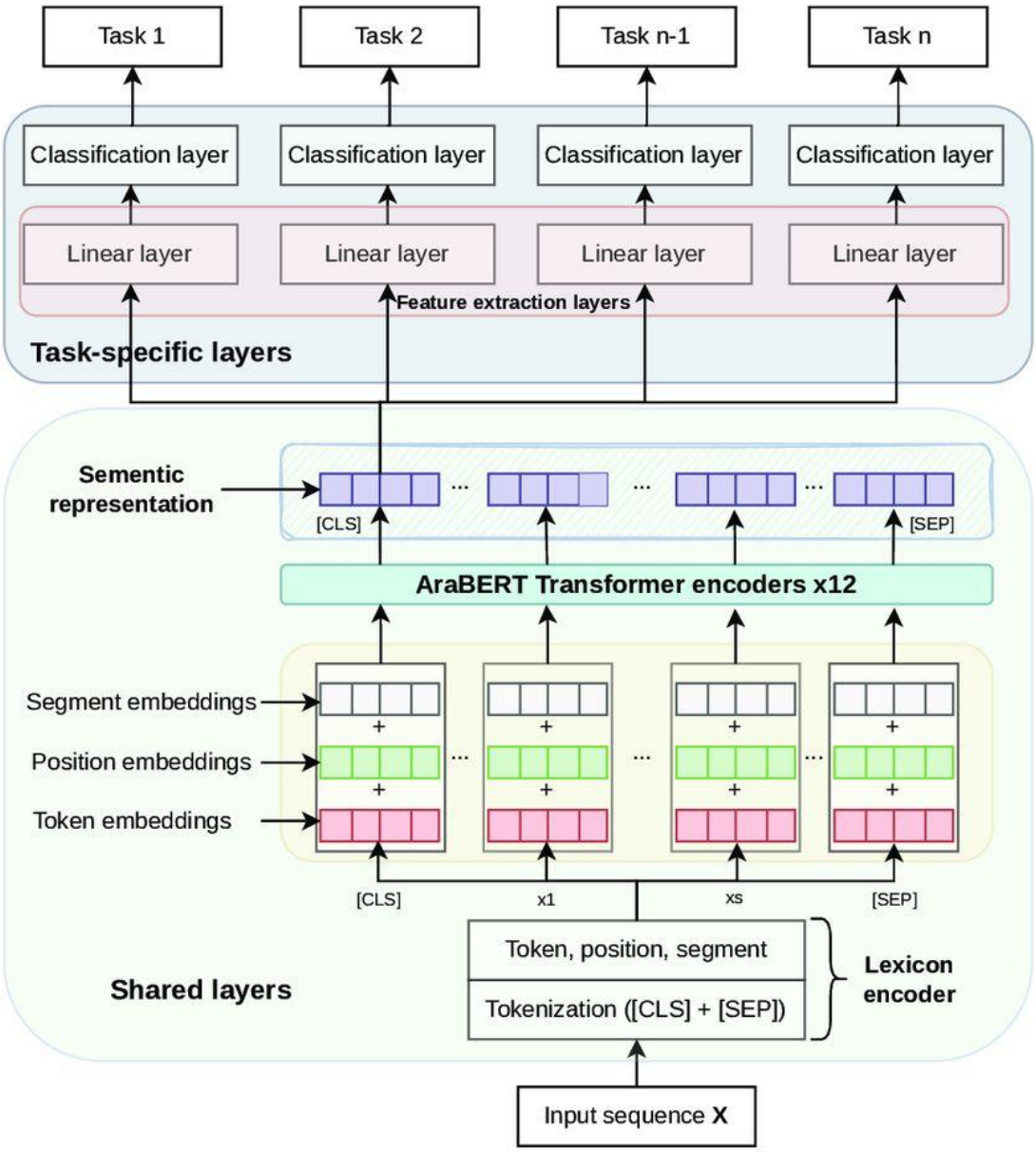
AraBERT Architecture Overview

AraBERT is based on the transformer architecture, a deep learning model that uses layers of self-attention mechanisms to capture the relationships between words in a given sentence. The model utilizes multiple transformer layers that process the input text in parallel, enabling the model to learn both local and global relationships between words. The output of each transformer layer is passed through a classification layer where each token is assigned a label indicating the named entity type (e.g., person, location, organization, etc.). This structure makes AraBERT particularly effective for sequence labeling tasks such as NER**.**

Why We Chose AraBERT for Political NER?

For political NER tasks, the ability to capture relationships between entities and contextualize them within the broader discourse is essential. AraBERT's deep bidirectional attention mechanism allows the model to consider both the left and right context of a word, enabling it to make better predictions in contexts where the relationship between entities is influenced by the surrounding text. This is crucial when identifying political figures, governmental institutions, events, and locations that are referenced in political speeches, articles, and social media posts.

As a result, AraBERT is not only well-suited to NER tasks in general, but its application in political discourse enables the efficient extraction of crucial information for governmental analysis, policy formation, and public opinion monitoring. Its robustness in handling complex Arabic text makes it an ideal model for this project, where accurate and context-aware entity recognition is pivotal**.**



• *The diagram illustrates the architecture of the AraBERT model, highlighting the shared layers (tokenization, embeddings, transformer encoders) and task-specific layers used for classification in .Named Entity Recognition (NER) tasks*

Training Procedures

The overall training procedure in our project involves fine-tuning the pre-trained AraBERT model with the preprocessed dataset using defined training arguments.

The details are:

Hyperparameters tuning
-output directory : set for model saving.
-evaluation strategy: evaluate after each epoch.
-Learning rate: experiment with different values to find the optimal performance (e.g 1e-5,2e-5,3e-5).
-Number of epochs: 3 or more.
-Weight decay: 0.01
-Batch size: is dynamically managed using DatacollatorForTokenClassific.

Optimization techniques
AdamW optimizer is used with weight decay equal to 0.01 , it helps to prevent overfitting and ensure better generalization. In addition there will be a linear learning rate scheduler with a warmup to smooth the convergence , also Gradient clipping are incorporated to stabilize the training.

Trainer
Hugging Face's Trainer API is used to manage the training process.
The trainer is initialized with the model , training arguments , datasets (train and validation sets) , data collator ,tokenizer and evaluation metrics.

Evaluation Metrics

To evaluate the performance of our fine-tuned AraBERT model for Named entity recognition (NER) on political text, we employ standard NER evaluation metrices. These metrices ensure a comprehensive understanding of model's ability to correctly recognize entities in Arabic political discourse.

The primary metrics include:

Accuracy

which measures the proportion of correctly identified entities out of all prediction

Precision

Measures how precise/accurate the model is. This metric reveals how many of the predicted entities are correctly labeled.
Precision is essential to ensure the model does not over-predict entities or assign

labels incorrectly to non-entity tokens. In the context of BIO tagging, this helps maintain labeling integrity and reduces the number of false positives.

Recall

Measures the model's ability to predict actual positive classes. This metric reveals how many of the predicted entities are correct.

High recall indicates that the model is effectively capturing all relevant entities in the input text. This is particularly crucial in political discourse, where failing to detect key names, organizations, or agreements may lead to incomplete information extraction.

F1-Score

The harmonic mean of precision and recall, providing a balanced assessment of model performance. is a function of Precision and Recall. It's needed when you seek a balance between Precision and Recall.
The F1-score considers both the correctness and completeness of predictions, making it a reliable overall metric for NER tasks

Token-Level Accuracy

Represents the percentage of tokens (individual word-pieces) for which the model correctly predicts the corresponding BIO label.
Since preprocessing involves tokenization and padding, token-level accuracy gives insight into how accurately the model handles each token. However, while useful, it may not fully reflect the model's ability to identify full entity spans.

Entity-Level F1 Score (Exact Match)

Evaluates the model's ability to identify entire named entities with both the correct span and correct label.
This metric ensures that multi-token entities, such as "League of Arab States", are fully and correctly labeled. In high-stakes applications like political text analysis, correct entity boundary detection is vital for meaningful interpretation and downstream decision-making.

Transition to Implementation Phase

The following section marks the transition from the planning and setup stages to the actual implementation of the proposed NER model using AraBERT on Arabic political texts.

Computational Environment

The implementation and evaluation of the Named Entity Recognition (NER) model in this project were carried out using both a local machine and the Google Colab platform, depending on the stage and computational requirements of each task.

Initially, data preprocessing, cleaning, and preliminary experiments were performed on a local machine. The device used is a Lenovo laptop equipped with an Intel Core i7 processor and 16 GB of RAM. The operating system is Windows, and the development environment includes Python version 3.10. Key libraries used during this phase include pandas, numpy, and matplotlib for data manipulation and visualization, along with transformers and torch for working with the pre-trained AraBERT model.

For model training and evaluation, we utilized Google Colab, a cloud-based platform that provides a flexible environment for machine learning projects. Due to limitations with GPU compatibility during visualization and result rendering, the CPU runtime was selected. The CPU provided by Colab is an Intel Xeon processor running at 2.20 GHz, with approximately 12.72 GB of available RAM and temporary disk storage of about 100 GB. Despite not using GPU acceleration, the CPU-based environment was sufficient for fine-tuning the model on our specific dataset size.

The Colab environment was configured with Python 3.10 and included the necessary libraries for deep learning and evaluation: transformers, torch, seqeval (for NER-specific evaluation metrics), scikit-learn, and arabert from CAMeL Lab. This setup enabled us to train the model, extract entities from political texts, and assess performance using standard evaluation metrics.

Data Ingestion

Ensuring data integrity was a critical component of our Named Entity Recognition (NER) project. From the moment the dataset was ingested until it was tokenized and prepared for training, we applied a series of deliberate and systematic steps to guarantee that the data used was clean, consistent, and structurally valid for fine-tuning the AraBERT model.

1.Data Loading and Verification

The dataset was sourced from the iSemantics/conllpp-ner-ar repository on Hugging Face. It includes token-level annotations following the BIO format, which is essential for NER tasks. Upon loading, we verified the data format, ensured that no columns were missing, and confirmed the integrity of token-label alignment.

2.Text Cleaning

To maintain quality, we performed essential text cleaning tasks. This included:

Removing special characters, numbers, and excessive punctuation

Eliminating unnecessary white spaces

Standardizing Arabic text to reduce variations in spelling or script forms (e.g., normalizing Alef forms)

This preprocessing ensured that irrelevant noise was removed from the data, thus enhancing the quality of input fed into the tokenizer.

3. Tokenization with AraBERT Tokenizer

Given the complexity of Arabic morphology, we used the AraBERT tokenizer to segment the cleaned text into subword tokens. This step was crucial for preserving semantic and syntactic context, as AraBERT relies on subword units for handling out-of-vocabulary words and rich morphological structures.

4.Label Alignment and Encoding

Since tokenization can split a single word into multiple subword units, we aligned the BIO entity labels accordingly. This ensured that every token — whether a word or subword — retained the correct entity label. Tokens not carrying entity information were marked with 'O.'

5. Data Splitting

To preserve data integrity during training and evaluation, we split the dataset into three distinct sets:

Training Set: 10,250 examples

Validation Set: 2,383 examples

Test Set: 2,572 examples

Each split was stratified and balanced to ensure entity types were adequately represented.

6. Padding, Truncation, and Attention Masks

All input sequences were padded or truncated to a uniform length, enabling batch training without structural inconsistencies. Attention masks were also applied to guide the model to focus only on valid tokens while ignoring padded elements.

By following these practices, we maintained high data integrity throughout our pipeline, ensuring that the model was trained on reliable, well-prepared data. This contributed significantly to the robustness and accuracy of our NER predictions on political Arabic text.

Model Initialization

In this project, we selected AraBERT as the primary model for handling Arabic texts in the Named Entity Recognition (NER) task, particularly in political texts. AraBERT is a transformer-based model that has been pre-trained on a large Arabic corpus, making it especially suitable for understanding the complex structures of the Arabic language. The model leverages a bidirectional attention mechanism, which is a key feature of the transformer architecture, allowing it to process text in both left-to-right and right-to-left directions. This is essential for understanding the relationships between words in a given context, which is particularly crucial in political texts where the meaning of entities such as political figures, organizations, and locations can change based on their surrounding context.

The pre-trained AraBERT model was loaded from the Hugging Face platform, where it was made available for fine-tuning on specific tasks. By using this pre-trained version, we were able to benefit from the deep understanding of Arabic language structures that AraBERT provides. However, this pre-trained model was not sufficient for our task, as political texts contain specialized terminology and context-specific references that require further adaptation.

To tailor AraBERT for our specific task, we performed fine-tuning on the model using a specialized dataset of political texts. This fine-tuning process allows the model to adjust its internal weights to better recognize entities in political discourse, including people (PER), organizations (ORG), locations (LOC), and other miscellaneous entities (MISC). By fine-tuning the model on labeled political data, AraBERT was trained to learn the specific characteristics of political language, improving its accuracy and performance in recognizing entities relevant to the political domain.

The fine-tuning process involved training the pre-trained model on a labeled dataset containing political texts with named entity annotations. This allowed the model to better understand the nuances of political language, such as references to governmental figures, institutions, and international agreements, which are critical for decision-making and information processing in political contexts. The fine-tuning process was essential in ensuring that the model could adapt to the specific challenges presented by political texts and the complexities of named entity recognition in this domain.

AraBERT's ability to understand the intricacies of the Arabic language, combined with its fine-tuning on political texts, ensured that the model could accurately identify and classify key entities in political discourse. This step was fundamental to achieving high performance in the NER task, as it allowed the model to handle the specific challenges posed by political language, such as ambiguity and context-dependent meanings, ensuring reliable and accurate results.

Training and Evaluation

The overall training procedure in our project involves fine-tuning the pre-trained AraBERT model with the preprocessed dataset using defined training arguments.

Training Process:

1- Loading Dataset.
2- Cleaning Data.
3-Tokenization.
4-Model Setup.
5-Training Arguments:
The training configuration is set using the TrainingArguments class include:

- Batch size of 16 for both training and evaluation.
- 5 epochs of training.
- Weight decay of 0.01 to prevent overfitting.
- Mixed precision (FPIo) to speed Up training on GPUs.
- Logging and checkpoint saving at every 10 and 500 steps respectively.

6-Trainer Initialization:
The Trainer class is initialized with the model, training arguments, training and validation datasets, tokenizer, and metrics computation function.
7-Training the model:
The training process starts with the trainer. train() method, which begins training the model using the training data and logs the results.

Training Results & Analysis:

Training Loss:

- Training Loss: Decreased from 1.3574 to 0.0141 after 3200 steps.

evaluation Metrices on Validation Set:

- Loss: 0.2092
- Accuracy: 95.36%
- F1 Score: 0.8491
- Precision: 0.8677
- Recall: 0.8347

Classes Performance:

- Class "O" : F1 Score: 0.980 ,precision: 0.977, recall:0.983 and support:31273.
- Class "B-PER" : F1 Score: 0.933 , precision: 0.91 ,recall:0.96 and support:2585.

- Class "B-ORG" : F1 Score: 0.89 precision: 0.9, recall:0.89 and support:3013.
- Class "B-LOC" : F1 Score: 0.866 , precision: 0,87 , recall:0.86 and support:1646.
- Class "B-MISC" : F1 Score: 0.73, precision: 0.77 c recall:0.69 and support:684.
- Class "I-PER" : F1 Score: 0.95, precision: 0.95 , recall:0.95 and support:2381.
- Class "I-ORG" : F1 Score: 0.84 ,precision: 0.8 , recall:0.8 and support:1574.
- Class "I-LOC" : F1 Score: 0.77, precision: 0.76 ,recall:0.79 and support:233.
- Class "I-MISC" : F1 Score: 0.677, precision: 0.79 , recall:0.59 and support:357.

Analysis and Suggestion:

- The performance on common entity classes like Person, Organization, and Location is strong.
- Challenges with miscellaneous entities (I-MISC) with low recall.
- We suggest to expand the dataset especially the I-MISC class.

Testing and Evaluation

After we train the model on the Arabic NER dataset, we evaluated its performance using the validation set. The evaluation was based on commonly used evaluation metrics in NER tasks: Accuracy, Precision, Recall, F1 Score, and Loss. Below is a detailed analysis of the results.

Evaluation Metrics:

- Accuracy: 95.44%
- F1 Score: 85.43%
- Precision: 87.18%
- Recall: 84.10%
- Loss: 0.2123

Analysis:

- Accuracy =95.44% which indicates that most of the tokens were correctly labeled. However, since this includes a large number of 'O' (non-entity) tokens, it is not the best indicator for NER performance.
- Precision =87.18% shows that the model made mostly correct predictions when it decided a token is part of a named entity. This reduces the number of false positives.
- Recall =84.10% reflects the model's ability to detect most of the actual named entities. A high recall means fewer missed entities.
- F1 Score =85.43% provides a balanced measure between precision and recall. It is the most important metric in NER tasks, and a score above 85% indicates strong and reliable model performance.

- Loss =0.2123= is relatively low, indicating that the model learned effectively and generalized well to unseen validation data.

Entity-wise Performance:

- The model performed best on the PER (Person) and ORG (ORGANIZATION) entities. This is likely because these entities were the most frequent in the dataset.
- The MISC(Miscellaneous) entity type showed the lowest performance due to fewer examples available in the training data.

Model Saving

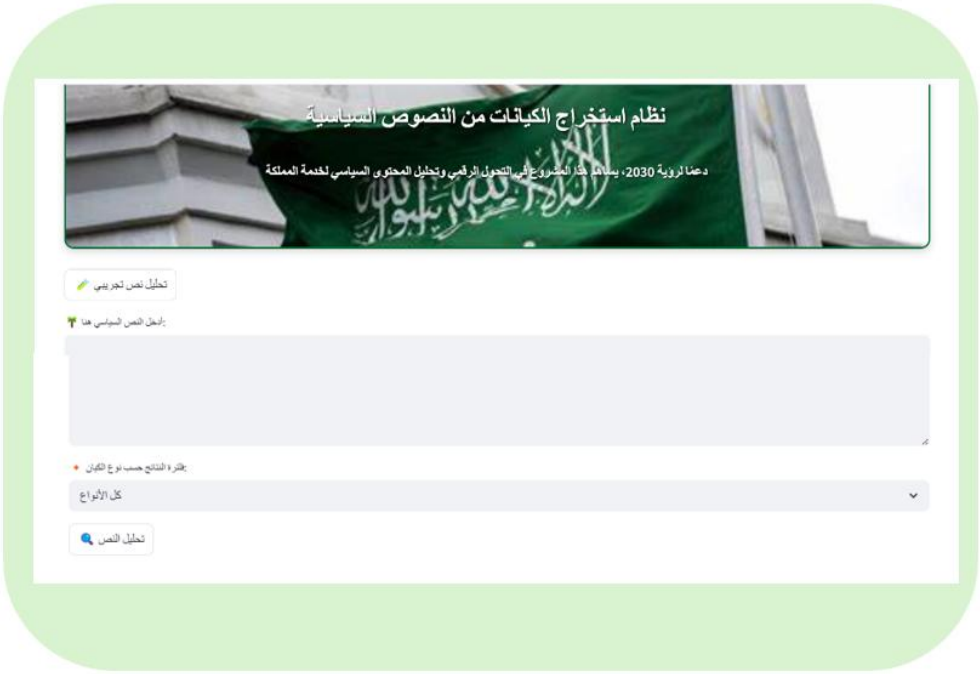
In the model saving phase, the final trained model was stored in a reusable format using the torch.save() function. The model was saved as .pth file, which allows for easy loading and inference without the need for retraining. Checkpointing was implemented to save the best-performing model during training based on evaluation metrics. The saved model is organized under a designated directory structure for easy access and management.
The final model size was approximately 1.4 GB, which is suitable for deployment on systems with moderate resources.

Interface and Deployment

To make the trained model accessible and usable, a simple web-based interface was developed. This interface was deployed using a lightweight web server and made publicly accessible via the Ngrok tunneling service.

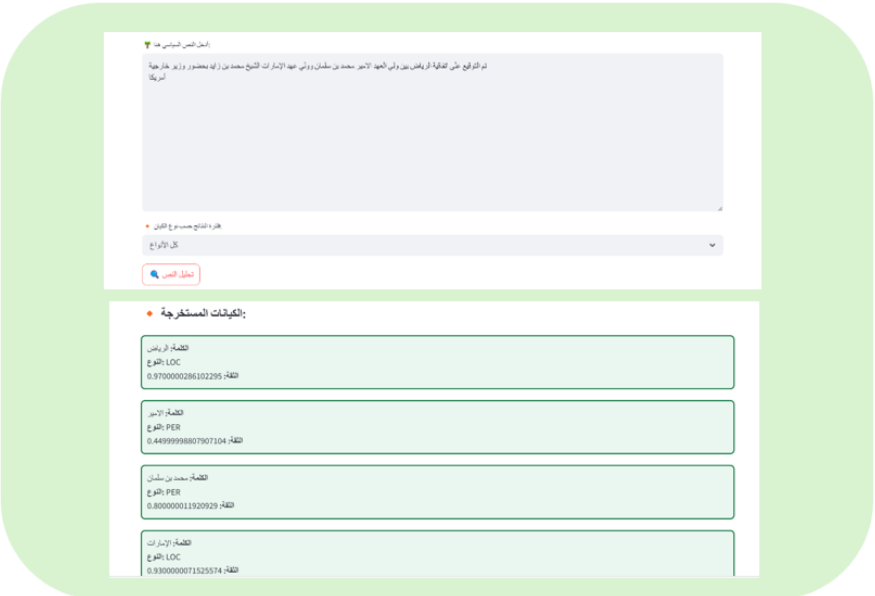
Ngrok enabled temporary external access to the local server, allowing for easy sharing and demonstration of the model's capabilities without the need for hosting on a dedicated cloud platform.

The interface receives text input from users and displays the model's predicted named entities in a structured output. This design ensures clarity and ease of use, particularly during the testing and demonstration phases of development.



Example Output from the User Interface:

To assess the functionality and clarity of the developed interface, two main testing scenarios were conducted:



Group 1 – Full Extraction Without Filtering:

In this scenario, a political text was input into the interface. The system extracted all detected named entities across various categories (e.g., persons and locations) and displayed them along with their associated confidence scores.



Group 2 – Filtered Results:
In this scenario, the user first applied a filter for the "Organization" (ORG) category. Since the input text did not contain any organizational entities, the system correctly displayed a message indicating that no relevant entities were found. Subsequently, the user applied a filter for the "Person" (PER) category, and the interface successfully isolated and displayed the person-related entities.

Results Analysis

The evaluation of the NER model on both the validation and test sets demonstrates robust performance, particularly in detecting named entities such as persons (PER), locations (LOC), and organizations (ORG). On the validation set, the model achieved a high accuracy of 95.36%, with an F1-score of 84.91%, indicating a strong balance between precision (86.77%) and recall (83.47%). Notably, the model excelled in detecting "O", "B-PER", and "I-PER" entities, all scoring above 93% F1.

When tested on unseen data (test set), the model maintained strong generalization with an accuracy of 94.38% and an F1-score of 82.26%, which suggests consistent performance and minimal overfitting. Although there was a slight drop in precision and recall, this decline remains within acceptable thresholds. Lower scores in the "B-MISC" and "I-MISC" categories highlight areas for potential improvement, possibly due to class imbalance or ambiguous context.

A practical test using a sample Arabic sentence further confirmed the model's effectiveness, successfully extracting entities such as *"سلمان بن عبدالعزيز"* (PER), *"الشرق الأوسط"* and *"القاهرة"* (LOC), and *"العربية"* (MISC) with high confidence scores ranging from 0.86 to 0.99. These results underscore the model's applicability in real-world scenarios where entity extraction is essential.

Summary Table of Evaluation Metrics:

Dataset	Accuracy	Precision	Recall	F1-Score	Loss
Validation Set	95.36%	86.77%	83.47%	84.91%	0.2092
Test Set	94.38%	81.80%	82.77%	82.26%	0.2859

Conclusion

In this project, a Named Entity Recognition (NER) system was developed using the AraBERT model, aimed at extracting named entities from political Arabic texts to support decision-making and information processing in political and governmental contexts. By leveraging a curated dataset of diplomatic messages and political content, the model was fine-tuned to identify entities such as people, locations, organizations, and miscellaneous elements with high precision.

The training and evaluation processes revealed strong overall performance, with the model achieving over 95% accuracy on the validation set and maintaining generalizability with a 94% accuracy on the test set. These results confirm the model's ability to automate the recognition of key information in Arabic political text, significantly reducing the manual workload typically involved in analyzing such documents.

The final implementation included a user interface that enables users to input political statements and receive real-time named entity predictions, making the system practical for integration into digital analysis tools used by researchers, analysts, and decision-makers.

Future Work

To build on the outcomes of this project, future work could focus on expanding the model's capabilities to support a broader range of political content such as news articles, social media discourse, or parliamentary records. Enhancing entity linking and relation extraction would also improve the depth of analysis.

Furthermore, incorporating deep learning techniques for political discourse analysis could allow the system not only to identify entities but also to interpret the intent of the speech, such as whether it is defensive, offensive, or neutral. This would significantly enrich political communication analysis and policy evaluation.

Additionally, integrating natural language generation (NLG) could allow the system to generate appropriate response speeches or official statements based on the analyzed content. This would serve as a practical aid for diplomats, spokespersons, and political writers, enabling timely and context-aware reactions to political messages.

By pursuing these future directions, the project has the potential to contribute to more comprehensive and intelligent political text analysis tools, amplifying the role of NLP and AI in shaping informed governance and policy-making.

References *(APA)*

1) Li, X., Wang, J., & Zhang, L. (2025). Named entity recognition for smart city data streams: Enhancing visualization and interaction. *Authorea Preprints*.
2) Ash, E., Gauthier, G., & Widmer, P. (2024). Relatio: Text semantics capture political and economic narratives. *Political Analysis, 32*(1), 115–132.
3) Guimarães, G. M., da Silva, F. X., Queiroz, A. L., Marcacini, R. M., Faleiros, T. P., Borges, V. R., & Garcia, L. P. (2024). DODFMiner: An automated tool for named entity recognition from official gazettes. *Neurocomputing, 568*, 127064.
4) SMT Center. (n.d.). الدبلوماسية الإلكترونية: كيف يمكن للذكاء الاصطناعي تحسين العملية الدبلوماسية؟ *SMT Center*.
5) Antoun, W., Badr, M., & Hajj, H. (2021). AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP 2021)*.
6) Al-Khatib, M., & Durrani, N. (2020). Improving named entity recognition for Arabic using deep learning. In *Proceedings of the 2020 Conference on Arabic Natural Language Processing*.
7) Abd Elaziz, M., Dahou, A., Orabi, D. A., Alshathri, S., Soliman, E. M., & Ewees, A. A. (2023). A hybrid multitask learning framework with a Fire Hawk optimizer for Arabic fake news detection. *Mathematics, 11*(2), 258.
8) Antoun, W., Baly, F., & Hajj, H. (2020). AraBERT: Transformer-based model for Arabic language understanding. In *LREC 2020 Workshop Language Resources and Evaluation Conference* (pp. 9).
9) Microsoft Corporation. (n.d.). Evaluation metrics for custom named entity recognition. *Microsoft Azure AI Services Documentation Workshop on Custom NER Evaluation* (pp. 1–5).

Appendix

Here is an active link to the complete project code on Colab with the interface code:

https://colab.research.google.com/drive/1JVQ3PzgwrqJYuELz0fFmN3OUI-JfMNan?usp=sharing